# Stacks Revision

30 problems

Fast recall cards for the current stacks package. Read the problem name, picture the hook, then check the sample and bug magnets before opening the Java file. Exported in expanded card mode.

## #01 StackUsingAnArray

Easy

Fixed-capacity LIFO Easy Source

**Before reading: what does top represent when the stack is empty, full, and holding items?**

**PROBLEM DESCRIPTION**

Build a last-in-first-out collection backed only by a fixed-size integer array. Push adds a new top item, pop removes and returns the top item, peek reads it without removal, and invalid overflow or empty-stack operations must be rejected.

**INPUT**

```
capacity = 3
push(10), push(20), push(30)
peek(), pop(), pop(), pop()
```

**OUTPUT**

```
peek = 30
pops = 30, 20, 10
isEmpty = true
```

**TIME**
**O(1) per operation**

**SPACE**
**O(capacity)**

**DATA STRUCTURES**
**int[] array, top index**

**WHEN TO RECOGNIZE IT**

When you need to implement LIFO behavior directly and the maximum number of elements is known in advance.

**VISUAL HOOK**

top -> 30 | 20 | 10

push ↑ | pop ↓

1. Empty means top = -1.
2. Push checks space, increments top, then writes.
3. Pop reads top, then decrements it.

Treat top as the index of the newest item. Push moves top upward before writing; pop reads the newest item and moves top downward.

**CORE MOVE**

Keep top at -1 for an empty stack. Push writes at ++top after checking space. Pop returns stack[top--] after checking that an item exists.

**CODE SKELETON**

```
top = −1
push(x):
  if top >= capacity − 1: overflow
  stack[++top] = x
pop():
  if top == −1: underflow
  return stack[top−−]
```

**BRUTE FORCE / BASELINE**

Keep the newest item at array index 0. Push shifts every existing item right; pop removes index 0 and shifts the rest left. It works, but each push and pop becomes O(n).

**ALTERNATE APPROACHES**

- Use a resizable array so capacity grows automatically while push and pop remain amortized O(1).
- Use a linked list with its head as the top when a fixed capacity is undesirable.

**BUG MAGNETS**

- A full stack has top == capacity - 1, so check overflow before incrementing top.
- Never read stack[top] when top is -1; check empty before pop and peek.
- This implementation is fixed capacity and does not resize.

## #02 StackUsingLinkedList

Easy

Head-as-top LIFO Easy Source

**Before reading: why must every new node become the head?**

**PROBLEM DESCRIPTION**

Build a last-in-first-out collection using a singly linked list instead of an array. The newest value must be returned first, and push, pop, peek, and empty checks should work without a preset capacity.

**INPUT**

```
push(10), push(20), push(30)
peek(), pop(), pop(), pop()
```

**OUTPUT**

```
peek = 30
pops = 30, 20, 10
isEmpty = true
```

**TIME**
**O(1) per operation**

**SPACE**
**O(n)**

**DATA STRUCTURES**
**Singly linked list, head pointer**

**WHEN TO RECOGNIZE IT**

When you need LIFO behavior without a fixed capacity and can use the linked-list head as the newest-item boundary.

**VISUAL HOOK**

**head/top -> 30** **20** **10** **null**

**new.next = head** **head = new**

1. Create a node for the new value.
2. Point new.next to the current head.
3. Move head to new; pop reverses that boundary move.

The newest node always becomes head. Push links the new node before the old head; pop simply moves head to head.next.

**CORE MOVE**

For push, point the new node to the current head and replace head. For pop, save head's value and advance head to the next node.

**CODE SKELETON**

```
head = null
push(x):
  node = new Node(x)
  node.next = head
  head = node
pop():
  if head == null: underflow
  value = head.val
  head = head.next
  return value
```

**BRUTE FORCE / BASELINE**

Treat the linked-list tail as the stack top. Push can append there, but pop must walk from the head to find the node before the tail, making pop O(n) in a singly linked list.

**ALTERNATE APPROACHES**

- Use a doubly linked list with a tail pointer so both push and pop at the tail are O(1).
- Use an array-backed stack when compact memory and cache-friendly access matter more than flexible growth.

**BUG MAGNETS**

- Do not attach the new node after head; it must become the new head/top.
- Check for an empty head before peek or pop.
- Each node adds pointer overhead, but the stack grows without a fixed array capacity.

## #03 StackUsingQueues

Easy

Rotate newest to front Easy Source LeetCode

**Before reading: how many queue elements must rotate after adding the new top?**

**PROBLEM DESCRIPTION**

Implement a last-in-first-out stack while using only normal queue operations. The most recently pushed value must be returned by top and removed by pop, even though a queue normally exposes the oldest value first.

**INPUT**

```
push(10), push(20), push(30)
peek(), pop(), pop(), pop()
```

**OUTPUT**

```
peek = 30
pops = 30, 20, 10
isEmpty = true
```

**TIME**
**Push O(n); others O(1)**

**SPACE**
**O(n)**

**DATA STRUCTURES**
**Single queue**

**WHEN TO RECOGNIZE IT**

When you must build LIFO behavior using only FIFO queue operations and can pay extra work during push.

**VISUAL HOOK**

**offer 30** **10** **20**

**rotate old items** **30** **20** **10**

1. Offer the new value at the queue back.
2. Rotate exactly the older elements from front to back.
3. The new value reaches the front, so poll now behaves like pop.

Push places the new value at the back, then rotates every older value behind it. The newest value finishes at the queue front.

**CORE MOVE**

Offer the new value, then move exactly the previous queue-size elements from front to back. After rotation, poll and peek behave like stack pop and top.

**CODE SKELETON**

```
push(x):
  queue.offer(x)
  repeat queue.size() - 1 times:
    queue.offer(queue.poll())
pop():
  if queue empty: underflow
  return queue.poll()
```

**BRUTE FORCE / BASELINE**

Keep normal queue order during push. For every pop, move the first n - 1 elements into a second queue, remove the final element, then swap the queues. Push is O(1), but pop is O(n).

**ALTERNATE APPROACHES**

- Use two queues and choose whether push or pop should carry the O(n) work.
- Use one queue as shown and rotate after push so both peek and pop stay O(1).

**BUG MAGNETS**

- Rotate queue.size() - 1 items, not the full size; one extra rotation restores FIFO order.
- Check empty before poll or peek to avoid null-to-int unboxing errors.
- This design makes push O(n) so pop and peek can stay O(1).

## #04 ValidParanthesis

Easy

Match the latest opener Easy Source LeetCode

**Before reading: why does a closing bracket compare only with the stack top?**

**PROBLEM DESCRIPTION**

Given a string made only of (), [], and {}, return whether it is valid. Every opener must be closed by the same bracket type, and nested brackets must close in the reverse order in which they opened.

**INPUT**

```
s = "{[]}"
```

**OUTPUT**

```
true
```

**TIME**
**O(n)**

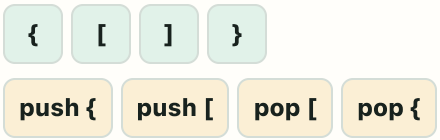
**SPACE**
**O(n)**

**DATA STRUCTURES**
**Stack of characters**

**WHEN TO RECOGNIZE IT**

When nested symbols must close in reverse order, so the latest unmatched opener must be checked first.

**VISUAL HOOK**




1. Push every opening bracket.
2. A matching closer removes the latest opener.
3. Any mismatch stays unresolved; success requires an empty stack.

Openers pile onto the stack. Every closer must match the opener currently on top; a mismatch remains unresolved.

**CORE MOVE**

Scan left to right. Push opening brackets. When a matching closer arrives, pop the top opener. The string is valid only when the stack ends empty.

**CODE SKELETON**

```
stack = empty
for c in s:
  if c closes stack.peek(): stack.pop()
  else: stack.push(c)
return stack.isEmpty()
```

**BRUTE FORCE / BASELINE**

Repeatedly remove every (), [], and {} pair from the string until nothing changes. If the final string is empty it is valid, but repeated rebuilding can take $O(n^2)$.

**ALTERNATE APPROACHES**

- Use a Deque instead of the legacy Stack class; push expected closing brackets so each closer needs only one comparison.
- Store closer-to-opener pairs in a map to make the matching logic shorter and easier to extend.

**BUG MAGNETS**

- A closer can match only the current stack top, not any earlier opener.
- A closing bracket with an empty stack is invalid.
- The final stack must be empty; otherwise some opening brackets never closed.

## #05 ReverseUsingStacks

Easy

Pop into reverse order Easy Source

**Before reading: why does popping characters reverse their original order?**

**PROBLEM DESCRIPTION**

Given a string, return another string containing exactly the same characters in reverse order. This exercise specifically asks you to use stack behavior so the last character read becomes the first character returned.

**INPUT**

```
s = "Arvind Kasale"
```

**OUTPUT**

```
"elasaK dnivrA"
```

**TIME**
**O(n)**

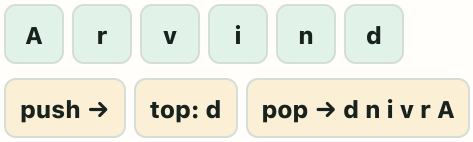
**SPACE**
**O(n)**

**DATA STRUCTURES**
**Stack of characters, char[] output**

### WHEN TO RECOGNIZE IT

When items must come back in exactly the opposite order from how they were read.

### VISUAL HOOK




1. Push characters from left to right.
2. The final character becomes the stack top.
3. Pop into the output from index 0 onward.

Read left to right and stack the characters. Popping removes the last character first, so writing each pop builds the reversed string.

### CORE MOVE

Push every input character. Then repeatedly pop the stack and write each character into the next output position.

### CODE SKELETON

```
stack = empty
for c in s:
  stack.push(c)
out = char[s.length]
i = 0
while stack not empty:
  out[i++] = stack.pop()
return String(out)
```

### BRUTE FORCE / BASELINE

Build a new string by prepending each character. The result is simple, but immutable string rebuilding makes it $O(n^2)$.

### ALTERNATE APPROACHES

- Swap characters from both ends of a char array with two pointers for O(n) time and no stack.
- Use StringBuilder.reverse() when the task allows library helpers.

### BUG MAGNETS

- The output index must advance once for every pop.
- Keep popping until the stack is empty so no character is skipped.
- The stack and output array both use O(n) extra space.

## #06 RemoveAllAdjacentDuplicates

Easy

Cancel matching neighbors Easy Source LeetCode

**Before reading: why must the current character compare only with the latest survivor?**

### PROBLEM DESCRIPTION

Given a lowercase string, repeatedly delete adjacent pairs containing the same letter. Each deletion may bring another equal pair together; return the final string after no duplicate neighbors remain.

**INPUT**

`s = "azxxzy"`

**OUTPUT**

`"ay"`

**TIME**

**O(n)**

**SPACE**

**O(n)**

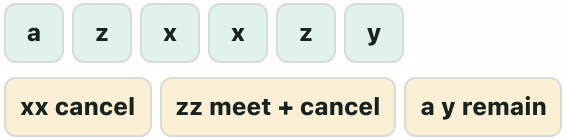
**DATA STRUCTURES**

**Stack of characters, StringBuilder**

**WHEN TO RECOGNIZE IT**

When equal neighbors cancel and each cancellation can expose a new matching pair behind them.

**VISUAL HOOK**




1. Push a character when it differs from the stack top.
2. Pop when it matches the top; the adjacent pair disappears.
3. The next input character now sees the newly exposed survivor.

The stack top is the surviving character immediately before the current one. Equal means cancel the top; different means keep the new character.

**CORE MOVE**

Scan once. If the current character equals the stack top, pop the top pair away. Otherwise push the character. The remaining stack is the answer in original order.

**CODE SKELETON**

```
stack = empty
for c in s:
  if stack not empty and stack.top == c:
    stack.pop()
  else:
    stack.push(c)
return characters from stack bottom to top
```

**BRUTE FORCE / BASELINE**

Scan the string, delete one adjacent duplicate pair, then restart scanning because the deletion may create a new pair. Repeated string rebuilding can take $O(n^2)$.

**ALTERNATE APPROACHES**

- Use a StringBuilder as the stack: compare with its last character, delete on a match, and append otherwise.
- Use a char array plus a write index as an in-place stack for O(n) time with lower overhead.

**BUG MAGNETS**

- After a pair disappears, the next character must compare with the newly exposed stack top.
- Iterate the final stack from bottom to top; popping it would reverse the surviving characters.
- Use Stack<Character> with the generic type on both sides to avoid a raw-type warning.

# #07 MinimumRemoveToMakeValidParanthesis

Medium

Drop impossible parentheses Medium Source LeetCode

**Before reading: which invalid parentheses can be identified immediately, and which must wait until the end?**

**PROBLEM DESCRIPTION**

Given a string containing letters plus '(' and ')', remove the fewest parentheses needed to make the parentheses valid. Return any valid result while preserving every letter and the order of all characters that remain.

**INPUT**

`s = "((a)b(c)d"`

**OUTPUT**

`"((a)bc)d"`

**TIME**
**O(n)**

**SPACE**
**O(n)**

**DATA STRUCTURES**
**StringBuilder, balance counter**

**WHEN TO RECOGNIZE IT**

When only parenthesis validity matters and unmatched closers can be rejected immediately, while leftover openers can be removed from the right.

**VISUAL HOOK**

( ( a ) b ( c ) d

**keep matched pairs** **1 '(' remains**

**remove from right side**

1. Scan left to right and skip any ')' with no available opener.
2. The remaining balance counts extra '(' characters.
3. Scan from right to left and skip that many '('.

Left to right, keep a ')' only when an earlier '(' is available. After that pass, any remaining balance represents extra '(' characters; remove those from right to left.

**CORE MOVE**

Build a first-pass string while skipping unmatched closing parentheses. Reverse it, skip exactly the remaining unmatched opening parentheses, then reverse once more to restore the original order.

**CODE SKELETON**

```
balance = 0
first = empty
for c in s:
  if c == '(':
    balance++; keep c
  else if c == ')' and balance > 0:
    balance--; keep c
  else if c is letter:
    keep c
reverse first
skip balance opening parentheses
reverse result
```

**BRUTE FORCE / BASELINE**

Try removing each parenthesis, test every resulting string for validity, and continue until a valid result appears. This explores many combinations and quickly becomes exponential.

**ALTERNATE APPROACHES**

- Use a stack of indexes to mark unmatched parentheses, then build the answer while skipping those indexes.
- Use two forward passes with counts: first remove unmatched ')', then remove the required number of '(' while preserving a valid target count.

**BUG MAGNETS**

- A final balance alone is not enough; an unmatched ')' must be skipped at the moment it appears.
- With the balance-count approach, remove surplus '(' while scanning right to left; blindly removing from the left can delete an opener needed by an earlier ')'.
- Letters are never removed, and the relative order of retained characters must stay unchanged.

## #08 ValidParenthesisString

Medium

Use stars as flexible closers Medium Source LeetCode

**Before reading: why must the star used as a closing parenthesis appear after the unmatched opener?**

**PROBLEM DESCRIPTION**

Given a string containing '(', ')', and '*', return whether it can become valid. Each '*' may act as an opening parenthesis, a closing parenthesis, or an empty character.

**INPUT**

```
s = "((((*))))"
```

**OUTPUT**

```
true
```

**TIME**
**O(n)**

**SPACE**
**O(n)**

**DATA STRUCTURES**
**Stack of '(' indexes, stack of '*' indexes**

### WHEN TO RECOGNIZE IT

When wildcard characters can repair unmatched parentheses, but their positions determine whether they can legally close an earlier opener.

### VISUAL HOOK

**(** **(** **(** **(** ***** **)** **)** **)**

**normal ')' closes '(' first** *** becomes final ')'**

**star index must be after opener**

1. Store indexes of '(' and '*' in separate stacks.
2. A ')' consumes '(' first, then '*' only if needed.
3. Pair leftover '(' with later '*' indexes.

Use real closing parentheses first. After the scan, each leftover '(' needs a '*' located to its right so that star can act as its closing parenthesis.

### CORE MOVE

Store indexes of '(' and '*' separately. A ')' consumes an opener first, otherwise a star. Finally pair each leftover opener with a later star; fail when no such star exists.

### CODE SKELETON

```
opens = stack of indexes
stars = stack of indexes
for each index i:
  if '(': opens.push(i)
  else if '*': stars.push(i)
  else if opens not empty: opens.pop()
  else if stars not empty: stars.pop()
  else: return false
while opens not empty:
  require stars.pop() > opens.pop()
return true
```

### BRUTE FORCE / BASELINE

Try all three meanings for every '*', then validate each resulting parenthesis string. This is correct but grows exponentially with the number of stars.

### ALTERNATE APPROACHES

- Track the minimum and maximum possible open-parenthesis balance; this greedy range solves the problem in O(n) time and O(1) space.
- Use dynamic programming over string position and possible open counts when you want an explicit state-based formulation.

### BUG MAGNETS

- A star before an unmatched '(' cannot close that opener; compare their indexes.
- When reading ')', consume a real '(' before using a star.
- Any character other than '(' or ')' is treated as '*' by the current implementation.

# #09 DecodeString

Medium

Expand innermost brackets Medium Source LeetCode

**Before reading: what complete unit becomes available when a closing bracket appears?**

**PROBLEM DESCRIPTION**

Given an encoded string containing patterns like k[text], return the fully decoded string by repeating text exactly k times. Encoded groups may be nested, and repeat counts may contain multiple digits.

**INPUT**

```
s = "3[a2[c]]"
```

**OUTPUT**

```
"accaccacc"
```

**TIME**

**O(input + decoded output)**

**SPACE**

**O(input + decoded output)**

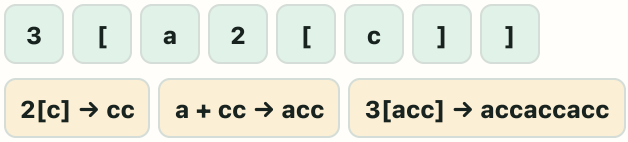
**DATA STRUCTURES**

**Stack of string tokens, StringBuilder**

**WHEN TO RECOGNIZE IT**

When closing a nested group requires finishing the most recently opened bracket before the outer group can continue.

**VISUAL HOOK**




1. Pop tokens until the nearest '[' to rebuild the inner text.
2. Pop every repeat-count digit immediately before that bracket.
3. Expand the text and push it back as one token.

Every ']' completes the nearest unmatched '['. Collapse that innermost group into one expanded token, push it back, and let any outer group consume it later.

**CORE MOVE**

Push characters as tokens. On ']', pop and rebuild the bracket contents, remove '[', pop the repeat-count digits, expand the contents, and push the expanded group back onto the stack.

**CODE SKELETON**

```
stack = empty
for token in encoded string:
  if token != ']':
    stack.push(token)
  else:
    text = pop until '['
    pop '['
    count = pop adjacent digits
    stack.push(text repeated count times)
return stack joined from bottom to top
```

**BRUTE FORCE / BASELINE**

Repeatedly search the whole string for an innermost bracket group and replace it with its expansion. It is understandable, but repeated scanning and immutable string rebuilding can become expensive.

**ALTERNATE APPROACHES**

- Use one stack for repeat counts and another for the partially decoded outer strings.
- Use recursive descent: parse characters until ']', recursively decode nested groups, then repeat the returned text.

**BUG MAGNETS**

- Repeat counts can have multiple digits, so collect every adjacent digit before converting the number.
- Characters popped from the stack arrive in reverse order and must be restored before expansion.
- An expanded inner group must be pushed back as one token so an outer group can repeat it.

## #10 EvaluateReversePolishNotation

Medium

Reduce operands on operators Medium Source LeetCode

**Before reading: which popped value belongs on the right side of subtraction or division?**

**PROBLEM DESCRIPTION**

Given arithmetic tokens written in Reverse Polish Notation, evaluate the expression and return its integer result. Numbers appear before their operator, every operator combines the two most recent available values, and integer division truncates toward zero.

**INPUT**

```
tokens = ["2", "1", "+", "3", "*"]
```

**OUTPUT**

9

**TIME**

**O(n)**

**SPACE**

**O(n)**

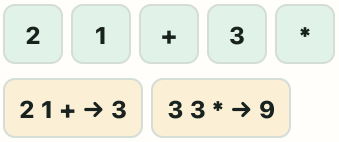
**DATA STRUCTURES**

**Stack of operands, operator set**

**WHEN TO RECOGNIZE IT**

When operators appear after their operands, so each operator can immediately reduce the two latest unresolved values.

**VISUAL HOOK**




1. Push every number token.
2. On an operator, pop right operand first and left operand second.
3. Compute left operator right and push the result.

Numbers wait on the stack. An operator removes the latest two values, computes one result, and pushes that result back as the next available operand.

**CORE MOVE**

Push number tokens. On an operator, pop the right operand first and the left operand second, calculate left operator right, then push the result.

**CODE SKELETON**

```
values = stack
for token in tokens:
  if token is number:
    values.push(token)
  else:
    right = values.pop()
    left = values.pop()
    values.push(apply(token, left, right))
return values.pop()
```

**BRUTE FORCE / BASELINE**

Repeatedly find an operator with two number tokens immediately before it, replace those three tokens with their result, and restart scanning. It works but repeated list rewriting can take $O(n^2)$.

**ALTERNATE APPROACHES**

- Use an integer stack instead of storing and reparsing numeric strings.
- Map each operator to a function so the reduction logic avoids a long conditional chain.

**BUG MAGNETS**

- Operand order matters for subtraction and division: the first pop is the right operand.
- The tokens are operators, not operands; name the operator set accordingly.
- Prefer Set.of("+", "-", "*", "/") over double-brace HashSet initialization.

## #11 SimplifyPath

Medium

Resolve directory segments Medium Source LeetCode

**Before reading: which path segments change the directory stack, and which disappear?**

**PROBLEM DESCRIPTION**

Given an absolute Unix-style path, return its canonical path. Collapse repeated slashes, ignore '.' segments, let '..' move to the parent directory when possible, and keep ordinary directory names unchanged.

**INPUT**

```
path = "/home/user///Documents/../Pictures"
```

**OUTPUT**

```
"/home/user/Pictures"
```

**TIME**

**O(n)**

**SPACE**

**O(n)**

**DATA STRUCTURES**

**Stack of directory names, StringBuilder**

**WHEN TO RECOGNIZE IT**

When path segments must be processed in order and '..' cancels the most recently retained directory.

**VISUAL HOOK**

**home** **user** **Documents** **..** **Pictures**

**push Documents** **.. → pop Documents**

**push Pictures**

1. Ignore empty segments and '.'.
2. '..' pops the latest retained directory when one exists.
3. Push normal names and join them beneath '/'.

The stack represents the current path from root to the active directory. A normal segment enters a directory; '..' backs out of the latest one.

**CORE MOVE**

Split on '/'. Ignore empty segments and '.', pop one directory for '..' when possible, and push every normal directory name. Join retained names beneath a single root slash.

**CODE SKELETON**

```
directories = stack
for segment in path.split('/'):
  if segment empty or segment == '.':
continue
  if segment == '..':
    if stack not empty: stack.pop()
  else:
    stack.push(segment)
return '/' + join(stack, '/')
```

**BRUTE FORCE / BASELINE**

Repeatedly replace //, /./, and directory/../ patterns inside the string until nothing changes. This is fragile and can require repeated O(n) rebuilding.

**ALTERNATE APPROACHES**

- Use a deque of directory names and join it at the end.
- Parse path segments manually with indexes to avoid creating the full split array.

**BUG MAGNETS**

- At the root, '..' does nothing; never pop an empty stack.
- A name containing dots, such as '...', is an ordinary directory name.
- Use && rather than & when combining boolean conditions so later checks short-circuit.

## #12 ExclusiveTimeOfFunctions

Medium

Pause and resume call frames Medium Source LeetCode

**Before reading: what timestamp should a paused caller resume from after an inclusive end event?**

**PROBLEM DESCRIPTION**

Given start and end logs from a single-threaded program, return how much time each function spends running by itself. Nested calls pause their caller, repeated calls with the same function ID still count toward that ID, and every end timestamp is inclusive.

**INPUT**

```
n = 2
logs = ["0:start:0", "0:start:2", "0:end:5",
"1:start:7", "1:end:7", "0:end:8"]
```

**OUTPUT**

```
[8, 1]
```

**TIME**
**O(logs)**

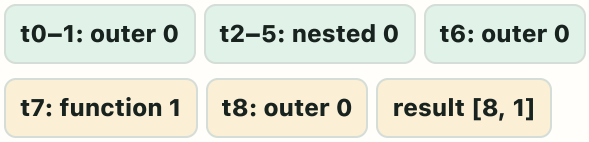
**SPACE**
**O(call depth + n)**

**DATA STRUCTURES**
**Stack of active call frames, result array**

**WHEN TO RECOGNIZE IT**

When nested start/end events describe a single active call at any moment, and each caller must resume exactly where its child finished.

**VISUAL HOOK**




1. At a start event, pay the current top only up to the child's start.
2. At an end event, pay the ending frame through that timestamp.
3. Resume the caller from end + 1.

The stack top owns the current time interval. A start event pays the previous top up to that timestamp, while an end event pays the top through the end timestamp and resumes its caller at end + 1.

**CORE MOVE**

Keep active start frames on a stack. On child start, add the elapsed time since the current frame's resume point, then push the child. On end, add end - resume + 1, pop the child, and set the caller's next resume point to end + 1.

**CODE SKELETON**

```
active = stack of frames
for log in logs:
  event = parse(log)
  if event is start:
    if active not empty:
      answer[top.id] += event.time -
top.resume
    active.push(event)
  else:
    frame = active.pop()
    answer[frame.id] += event.time -
frame.resume + 1
    if active not empty:
      top.resume = event.time + 1
```

**BRUTE FORCE / BASELINE**

Expand every timestamp into a timeline, determine which function is active at each unit, and increment that function's count. It is intuitive but can be impossibly slow when timestamps are large.

**ALTERNATE APPROACHES**

- Keep only function IDs on the stack plus one global previous timestamp; charge every interval to the current top.
- Store parsed log fields in lightweight records before processing when parsing clarity matters more than avoiding an extra pass.

**BUG MAGNETS**

- End timestamps are inclusive, so an ending call receives end - start + 1 time units.
- After a child ends, its caller resumes at end + 1, not at end.
- Nested calls may share the same function ID; stack frames, not IDs alone, represent active invocations.

## #13 ScoreOfParanthesis

Medium

Score each nested frame Medium Source LeetCode

**Before reading: why must every open parenthesis receive its own subtotal frame?**

**PROBLEM DESCRIPTION**

Given a balanced parentheses string, return its score. A primitive pair () scores 1, adjacent balanced groups add their scores, and wrapping a balanced group in parentheses doubles its score.

**INPUT**

`s = "()(()())"`

**OUTPUT**

5

**TIME**
**O(n)**

**SPACE**
**O(depth)**

**DATA STRUCTURES**
**Stack of subtotal scores**

### WHEN TO RECOGNIZE IT

When nested groups need their own subtotal because closing parentheses either create a primitive score or double only the score inside that group.

### VISUAL HOOK

**() → 1** **()() → 1 + 1 = 2**

**(()()) → 2 × 2 = 4** **()(()()) → 1 + 4 = 5**

1. Push zero when a new group opens.
2. On close, zero becomes 1; a non-zero subtotal becomes twice itself.
3. Add the completed group score to its parent frame.

Every '(' starts a fresh score frame at zero. A ')' completes that frame: zero means the primitive (), otherwise double the inner subtotal, then add it to the parent frame.

### CORE MOVE

Push zero for every opening parenthesis. On closing, pop the inner subtotal, turn it into 1 when empty or 2 × inner otherwise, then add that group score to the parent subtotal.

### CODE SKELETON

```
scores = stack containing 0
for c in s:
  if c == '(':
    scores.push(0)
  else:
    inner = scores.pop()
    group = inner == 0 ? 1 : 2 * inner
    parent = scores.pop()
    scores.push(parent + group)
return scores.pop()
```

### BRUTE FORCE / BASELINE

Repeatedly replace every primitive () with 1, then evaluate additions and doubled wrapped groups as an expression. This mirrors the rules but requires awkward repeated parsing and rebuilding.

### ALTERNATE APPROACHES

- Track nesting depth and add 2^depth whenever a primitive () closes, using O(1) extra space.
- Parse recursively: each call returns the score until its matching closing parenthesis.

### BUG MAGNETS

- Do not keep one global score and double it; wrapping must double only the current group's inner subtotal.
- An inner subtotal of zero means the closed group was exactly (), so its score is 1.
- After calculating a group, add it to the parent frame because adjacent groups sum.

# #14 MakeStringGreat

Easy

Cancel opposite-case neighbors Easy Source LeetCode

**Before reading: why is comparing the current character only with the latest survivor enough?**

**PROBLEM DESCRIPTION**

Given a string of uppercase and lowercase English letters, repeatedly remove adjacent characters that are the same letter in opposite cases, such as 'aA' or 'Bb'. Return the final string after no such adjacent pair remains.

**INPUT**

```
s = "leEeetcode"
```

**OUTPUT**

```
"leetcode"
```

**TIME**

**O(n)**

**SPACE**

**O(n)**

**DATA STRUCTURES**

**Stack of surviving characters, StringBuilder**

**WHEN TO RECOGNIZE IT**

When removing one adjacent pair can expose a new pair that must also be removed.

**VISUAL HOOK**

**l e E e e t c o d e** **e + E → remove**

**l e e e t c o d e**

**no more opposite-case neighbors → leetcode**

1. Keep surviving characters in a stack.
2. Pop when the current character and stack top are the same letter in opposite cases.
3. Otherwise push; the stack preserves the final order.

Treat the stack as the characters that have survived so far. The current character meets only the latest survivor; if they are the same letter with opposite cases, both disappear.

**CORE MOVE**

For each character, compare it with the stack top. Pop when both represent the same letter but have different cases; otherwise push the character. Read the surviving stack from bottom to top.

**CODE SKELETON**

```
survivors = stack
for c in s:
  if survivors not empty
     and lower(survivors.top) == lower(c)
     and survivors.top != c:
    survivors.pop()
  else:
    survivors.push(c)
return survivors from bottom to top
```

**BRUTE FORCE / BASELINE**

Scan the whole string for a bad adjacent pair, delete one pair, then restart the scan. It is easy to reason about but repeated string rebuilding can take $O(n^2)$ time.

**ALTERNATE APPROACHES**

- Use a StringBuilder as the stack: compare with its final character and delete the final character on a match.
- Use a character array plus a write index as an in-place stack, which keeps O(n) time with less object overhead.

**BUG MAGNETS**

- The letters must be equal ignoring case but different as actual characters; 'aa' and 'AA' stay.
- After a pop, the next character must compare with the newly exposed stack top.
- Build the result from stack bottom to top so the original survivor order is preserved.

## #15 MinimumStack

Medium

Carry the minimum with every value Medium Source LeetCode

**Before reading: what extra fact must each pushed value remember so getMin never scans the stack?**

**PROBLEM DESCRIPTION**

Design a stack that supports push, pop, reading the top value, and reading the current minimum value, with every operation taking constant time.

**INPUT**

```
push(-2), push(0), push(-3), getMin(), pop(), top(), getMin()
```

**OUTPUT**

```
-3, 0, -2
```

**TIME**

**O(1) per operation**

**SPACE**

**O(n)**

**DATA STRUCTURES**

**Stack of nodes storing value + minimum-so-far**

### WHEN TO RECOGNIZE IT

When a stack must answer an aggregate such as the current minimum instantly, even after values are popped.

### VISUAL HOOK

**push -2 → (-2, min -2)** **push 0 → (0, min -2)**

**push -3 → (-3, min -3)**

**pop -3 → top remembers min -2**

1. Read the previous minimum before pushing.
2. Push a node containing the value and the new minimum-so-far.
3. The top node answers both top and getMin; pop restores the previous snapshot.

Every node carries a snapshot of the minimum at the moment it was pushed. The top node therefore always knows the current minimum, and popping reveals the previous snapshot.

### CORE MOVE

On push, store both the new value and min(new value, previous top's minimum). Then top reads the value field, getMin reads the minimum field, and pop removes both together.

### CODE SKELETON

```
push(value):
  previousMin = stack empty ? value :
stack.top.min
  stack.push(Node(value, min(value,
previousMin)))
pop(): stack.pop()
top(): return stack.top.value
getMin(): return stack.top.min
```

### BRUTE FORCE / BASELINE

Store only values and scan the whole stack whenever getMin is called. Push and pop stay simple, but each minimum query costs O(n).

### ALTERNATE APPROACHES

- Maintain a normal value stack plus a second stack of minimums; push to the minimum stack when a new value is less than or equal to its top.
- Store encoded differences from the minimum in one numeric stack, allowing O(1) operations with less per-entry storage but trickier arithmetic.

### BUG MAGNETS

- Use the previous minimum when creating the new node; recomputing after push can accidentally read the new node itself.
- Duplicate minimum values must survive correctly after one copy is popped.
- The problem guarantees valid calls, but a reusable implementation should decide how empty pop, top, and getMin behave.

# #16 DailyTemperatures

Medium

Resolve colder days with a warmer day Medium Source LeetCode

**Before reading: why must the monotonic stack store day indexes instead of temperatures alone?**

**PROBLEM DESCRIPTION**

Given one temperature for each day, return how many days each day must wait for a strictly warmer temperature. If no warmer future day exists, leave its answer as zero.

**INPUT**

```
temperatures = [73, 74, 75, 71, 69, 72, 76, 73]
```

**OUTPUT**

```
[1, 1, 4, 2, 1, 1, 0, 0]
```

**TIME**

**O(n)**

**SPACE**

**O(n)**

**DATA STRUCTURES**

**Monotonic decreasing stack of indexes, result array**

**WHEN TO RECOGNIZE IT**

When every item needs the first greater value to its right, and one new greater value may answer several earlier items.

**VISUAL HOOK**

**stack: 75, 71, 69** **new day: 72**

**72 resolves 69 → wait 1** **72 resolves 71 → wait 2**

**75 stays unresolved**

1. Keep unresolved day indexes with decreasing temperatures.
2. A warmer current day repeatedly pops colder unresolved days.
3. For each popped day, write current index - popped index.

The stack holds indexes of days still waiting for warmth, with temperatures decreasing toward the top. A warmer current day repeatedly pops and resolves every colder day it can.

**CORE MOVE**

Scan days from left to right. While the current temperature is warmer than the temperature at the stack's top index, pop that index and store current index - popped index. Then push the current index.

**CODE SKELETON**

```
unresolved = stack of indexes
answer = zeros
for day from left to right:
  while unresolved not empty
        and temperatures[unresolved.top] <
temperatures[day]:
    colderDay = unresolved.pop()
    answer[colderDay] = day - colderDay
  unresolved.push(day)
return answer
```

**BRUTE FORCE / BASELINE**

For each day, scan forward until the first warmer day appears. This is straightforward but takes $O(n^2)$ time when many days have no warmer answer nearby.

**ALTERNATE APPROACHES**

- Scan from right to left and use already-computed waiting distances to jump over days that cannot be the answer.
- Because temperatures have a small bounded range, track the nearest future index for every possible temperature and search only warmer values.

**BUG MAGNETS**

- Store indexes, not only temperatures; the answer is the distance between day indexes.
- Use a while loop because one warmer day can resolve multiple colder days.
- Equal temperatures are not warmer, so pop only when stack-top temperature is strictly less than the current temperature.

## #17 SumOfSubarrayMinimums

Medium

Count each value's minimum territory Medium Source LeetCode

**Before reading: how many subarrays choose arr[i] as their minimum, and why must duplicate boundaries be asymmetric?**

**PROBLEM DESCRIPTION**

For every contiguous subarray, take its minimum value, then return the sum of all those minimums modulo 1,000,000,007.

**INPUT**

```
arr = [3, 1, 2, 4]
```

**OUTPUT**

17

**TIME**
**O(n)**

**SPACE**
**O(n)**

**DATA STRUCTURES**
**Two monotonic increasing stacks of indexes, left/right boundary arrays**

### WHEN TO RECOGNIZE IT

When summing a value across many subarrays is easier by counting how many subarrays choose each array element as their representative minimum.

### VISUAL HOOK

**value 1 at index 1**

**left boundary -1 → 2 start choices**

**right boundary 4 → 3 end choices**

**contribution: 1 × 2 × 3 = 6**

1. Find previous smaller-or-equal and next strictly smaller boundaries.
2. Multiply the number of valid start choices by valid end choices.
3. Add value × left choices × right choices using long arithmetic.

Each element owns a rectangle of subarrays: choose any start after its left boundary and any end before its right boundary. Left choices × right choices counts the subarrays where that occurrence is the chosen minimum.

### CORE MOVE

Find the previous smaller-or-equal index and next strictly smaller index for every element. Its contribution is value × (index - left boundary) × (right boundary - index). Sum contributions modulo MOD.

### CODE SKELETON

```
left[i] = previous index with value <= arr[i]
right[i] = next index with value < arr[i]
score = 0
for i:
  leftChoices = i - left[i]
  rightChoices = right[i] - i
  score += (long) arr[i] * leftChoices *
rightChoices
  score %= MOD
return score
```

### BRUTE FORCE / BASELINE

Generate every subarray, scan it for the minimum, and add that minimum. This directly follows the statement but can require $O(n^3)$ time, or $O(n^2)$ when maintaining a running minimum per start.

### ALTERNATE APPROACHES

- Use one monotonic stack with a sentinel and calculate an element's complete contribution when it is popped.
- Maintain a monotonic stack of value/count pairs and dynamically track the sum of minimums for all subarrays ending at the current index.

**BUG MAGNETS**

- Duplicates need asymmetric boundaries: one side accepts equal values and the other side does not, otherwise subarrays are double-counted or missed.
- Cast the value to long before multiplying; casting after int multiplication cannot repair overflow.
- Boundary indexes are outside the valid territory, so choices are index - left and right - index.

## #18 AsteroidCollision

Medium

Resolve head-on survivor chains Medium Source LeetCode

**Before reading: which direction pairing can actually collide, and when must the incoming asteroid keep fighting?**

**PROBLEM DESCRIPTION**

Asteroids appear in a line, where magnitude is size and sign is direction. When two asteroids moving toward each other collide, the smaller one disappears; equal sizes both disappear. Return the surviving asteroids in their original order.

**INPUT**

```
asteroids = [10, 2, -5]
```

**OUTPUT**

```
[10]
```

**TIME**
**O(n)**

**SPACE**
**O(n)**

**DATA STRUCTURES**
**Stack of surviving asteroids**

**WHEN TO RECOGNIZE IT**

When the current item may repeatedly destroy the latest survivors, but only one specific direction pairing can interact.

**VISUAL HOOK**

**survivors: [10, 2]** **incoming: -5**

**2 vs -5 → 2 breaks** **10 vs -5 → -5 breaks**

**survivor: [10]**

1. Only compare an incoming negative asteroid with a positive stack top.
2. Pop smaller positive tops and continue with the same incoming asteroid.
3. Push the incoming asteroid only if it survives every possible collision.

Only a right-moving survivor on the left and a new left-moving asteroid on the right are heading toward each other. The incoming asteroid keeps colliding with the stack top until one side survives or no collision is possible.

**CORE MOVE**

Push right-movers immediately. For an incoming left-mover, repeatedly compare it with a positive stack top: pop smaller tops, destroy both on equal size, or destroy the incoming asteroid when the top is larger. Push it only if it survives.

**CODE SKELETON**

```
survivors = stack
for asteroid in asteroids:
  incomingAlive = true
  while incomingAlive and asteroid < 0
        and survivors not empty and
survivors.top > 0:
    if survivors.top < abs(asteroid):
survivors.pop()
    else if survivors.top == abs(asteroid):
      survivors.pop(); incomingAlive = false
    else: incomingAlive = false
  if incomingAlive: survivors.push(asteroid)
return survivors in original order
```

**BRUTE FORCE / BASELINE**

Repeatedly scan adjacent asteroids, resolve the first head-on collision, rebuild the list, and restart. This is easy to simulate but repeated scans and deletions can take $O(n^2)$ time.

**ALTERNATE APPROACHES**

- Use an int array plus a write index as the survivor stack to avoid boxed Integer objects.
- Use an ArrayDeque<Integer> as the stack; it offers the same algorithm with a more modern Java stack API.

**BUG MAGNETS**

- Two asteroids collide only when the stack top is positive and the incoming asteroid is negative.
- After popping a smaller survivor, the incoming asteroid must continue fighting the newly exposed top.
- On equal magnitude, pop the top and mark the incoming asteroid destroyed so neither is pushed.

## #19 NextGreaterElementI

Easy

Precompute answers for subset queries Easy Source LeetCode

**Before reading: why compute answers for all of nums2 before answering the smaller nums1 query list?**

**PROBLEM DESCRIPTION**

Every value in nums1 also appears in nums2. For each nums1 value, return the first greater value appearing to its right in nums2, or -1 when none exists.

**INPUT**

```
nums1 = [4, 1, 2]
nums2 = [1, 3, 4, 2]
```

**OUTPUT**

```
[-1, 3, -1]
```

**TIME**
**O(n + m)**

**SPACE**
**O(n)**

**DATA STRUCTURES**
**Monotonic decreasing stack of values, HashMap lookup**

### WHEN TO RECOGNIZE IT

When next-greater answers belong to one reference array, but the requested output contains only a subset of its distinct values.

### VISUAL HOOK

**scan nums2 right → left**

**stack keeps greater candidates**

**1 sees 3 → bank[1] = 3**

**query nums1 through bank**

1. Scan nums2 right to left with greater candidates on a stack.
2. Store each distinct value's next-greater answer in a map.
3. Answer nums1 using direct map lookups.

Scanning from the right means the stack already represents possible answers to the right. Remove smaller values because the current value blocks them, then the surviving top is the nearest greater value.

### CORE MOVE

Build each nums2 value's next-greater answer with a right-to-left monotonic stack, store it in a map, then answer every nums1 query with one map lookup.

### CODE SKELETON

```
bank = map
candidates = stack
for value in nums2 from right to left:
  while candidates not empty and
candidates.top <= value:
    candidates.pop()
  bank[value] = candidates empty ? -1 :
candidates.top
  candidates.push(value)
return bank[value] for each value in nums1
```

### BRUTE FORCE / BASELINE

For every nums1 value, find it in nums2 and scan right until a greater value appears. This is simple but can take O(m × n) time.

### ALTERNATE APPROACHES

- Scan nums2 left to right and map answers when the current value pops smaller unresolved values.
- When queries are indexes rather than distinct values, store indexes in the stack and write directly into an answer array.

**BUG MAGNETS**

- The map works by value only because nums2 values are unique.
- Pop candidates that are not greater than the current value; equality would also be invalid if duplicates were allowed.
- The stack contains candidates from the right side only because the scan moves right to left.

## #20 NextGreaterElementIV2

Medium

Write next-greater answer per position Medium Source

**Before reading: which right-side values can be discarded forever when processing the current value?**

**PROBLEM DESCRIPTION**

For every element in an array, return the first strictly greater element appearing to its right. Return -1 for positions that have no greater element to their right.

**INPUT**

`arr = [13, 7, 6, 12]`

**OUTPUT**

`[-1, 12, 12, -1]`

**TIME**

**O(n)**

**SPACE**

**O(n)**

**DATA STRUCTURES**

**Monotonic decreasing stack of candidate values, result array**

**WHEN TO RECOGNIZE IT**

When every position needs the first greater value on its right and later values can be processed once as reusable candidates.

**VISUAL HOOK**

**scan: 12 ← 6 ← 7 ← 13**

**stack top = nearest usable greater**

**6 sees 12** **7 pops 6, then sees 12**

**13 pops all → -1**

1. Scan positions from right to left.
2. Pop every candidate less than or equal to the current value.
3. Write the remaining top or -1, then push the current value.

While scanning right to left, smaller or equal stack values can never answer the current element and are discarded. The remaining top is the first greater candidate that survived.

**CORE MOVE**

Scan from right to left. Pop every stack value less than or equal to the current value, write the remaining top or -1 as the answer, then push the current value.

**CODE SKELETON**

```
candidates = stack
answer = array
for i from last to first:
  while candidates not empty and
candidates.top <= arr[i]:
    candidates.pop()
  answer[i] = candidates empty ? -1 :
candidates.top
  candidates.push(arr[i])
return answer
```

**BRUTE FORCE / BASELINE**

For every index, scan all later elements until the first greater value appears. In a decreasing array, this takes $O(n^2)$ time.

**ALTERNATE APPROACHES**

- Scan left to right with a stack of unresolved indexes and fill their answers when a greater value arrives.
- Store indexes instead of values when later work needs positions or distances rather than only the greater value.

**BUG MAGNETS**

- Pop equal values too because the answer must be strictly greater.
- Right-to-left traversal makes the stack top the nearest surviving greater candidate.
- This implementation assumes the input list is non-empty when initializing its last element.

## #21 NextGreaterElementII

Medium

Unroll the circular next-greater scan Medium Source LeetCode

**Before reading: how can a circular right-side search become the same linear next-greater scan you already know?**

**PROBLEM DESCRIPTION**

For every element in a circular array, return the first strictly greater value encountered while moving right and wrapping back to the beginning. Return -1 when no greater value exists.

**INPUT**

`nums = [1, 2, 1]`

**OUTPUT**

`[2, -1, 2]`

**TIME**
**O(n)**

**SPACE**
**O(n)**

**DATA STRUCTURES**
**Doubled input array, monotonic decreasing stack of values, result arrays**

### WHEN TO RECOGNIZE IT

When a normal next-greater search must continue past the array's end and inspect values from the beginning once.

### VISUAL HOOK

**circle: [1, 2, 1]** **unroll: [1, 2, 1 | 1, 2, 1]**

**run next-greater on doubled view**

**keep answers for first copy only**

1. Unroll one circular lap by viewing the array twice.
2. Run the normal right-to-left decreasing-stack scan over the doubled view.
3. Keep only the first copy's answers.

Duplicating the array turns one circular lap into an ordinary linear right-side search. The second copy supplies wrapped candidates; only the first copy represents real output positions.

### CORE MOVE

Create two copies of nums, scan the doubled array from right to left with a decreasing candidate stack, and copy the first n next-greater answers into the result.

### CODE SKELETON

```
doubled = nums + nums
candidates = stack
next = array for doubled
for i from doubled.length - 1 to 0:
  while candidates not empty and
candidates.top <= doubled[i]:
    candidates.pop()
  next[i] = candidates empty ? -1 :
candidates.top
  candidates.push(doubled[i])
return next[0..n-1]
```

### BRUTE FORCE / BASELINE

For every position, move right for at most n - 1 steps using modulo indexes until a greater value appears. This is easy to understand but takes $O(n^2)$ time.

### ALTERNATE APPROACHES

- Avoid duplicated arrays by scanning indexes from 2n - 1 down to 0 and reading nums[i % n]; write answers only when i < n.
- Scan left to right twice with a stack of unresolved indexes, resolving them whenever a greater value appears.

**BUG MAGNETS**

- Equal values are not greater, so pop candidates less than or equal to the current value.
- The second copy exists only to provide wrapped candidates; return answers for the first n positions.
- A circular search may inspect each other element only once; it must not loop forever.

## #22 StockSpanner

Medium

Merge dominated price spans Medium Source LeetCode

**Before reading: why can one popped node contribute several days to today's span?**

**PROBLEM DESCRIPTION**

Process one stock price at a time. For each new price, return the number of consecutive days ending today whose prices were less than or equal to today's price.

**INPUT**

```
next calls: [100, 80, 60, 70, 60, 75, 85]
```

**OUTPUT**

```
[1, 1, 1, 2, 1, 4, 6]
```

**TIME**
**O(1) amortized per next call**

**SPACE**
**O(n)**

**DATA STRUCTURES**
**Monotonic decreasing stack of (price, compressed span) nodes**

**WHEN TO RECOGNIZE IT**

When an online stream asks how far backward the current value dominates consecutive earlier values, and previously solved ranges can be reused.

**VISUAL HOOK**

**stack: (80,1), (70,2), (60,1)** **new price: 75**

**pop (60,1) → span 2** **pop (70,2) → span 4**

**push (75,4)**

1. Begin today's span at one.
2. Pop every price less than or equal to today and inherit its compressed span.
3. Push today's price with the combined span for future calls.

Each node is a compressed block: its span already counts itself plus consecutive prices it dominated. When a higher price pops that node, it inherits the entire block at once.

**CORE MOVE**

Start the new price's span at 1. While the stack top price is less than or equal to it, pop the node and add its stored span. Push the new (price, total span) node and return the span.

**CODE SKELETON**

```
next(price):
  span = 1
  while stack not empty and stack.top.price
<= price:
    span += stack.pop().span
  stack.push(Node(price, span))
  return span
```

**BRUTE FORCE / BASELINE**

Store every historical price and scan backward from today until finding a greater price. A long increasing sequence makes repeated calls take $O(n^2)$ total time.

**ALTERNATE APPROACHES**

- Store day indexes in a decreasing stack and compute current day - previous greater day, requiring an explicit day counter.
- Use an ArrayDeque of price/span pairs for the same compressed-span algorithm with a modern Java stack API.

**BUG MAGNETS**

- Pop prices equal to today's price too because the span includes prices less than or equal to today.
- Add the popped node's full stored span, not just one day.
- The O(1) time is amortized: one call may pop many nodes, but each node is pushed and popped once.

## #23 MaximumWidthRamp

Medium

Match record-low starts with farthest ends Medium Source LeetCode

**Before reading: why are only strict record-low indexes useful as possible ramp starts?**

**PROBLEM DESCRIPTION**

Find the largest distance j - i such that i is before j and nums[i] is less than or equal to nums[j].

**INPUT**

`nums = [6, 0, 8, 2, 1, 5]`

**OUTPUT**

`4`

**TIME**

**O(n)**

**SPACE**

**O(n)**

**DATA STRUCTURES**

**Strictly decreasing monotonic stack of candidate start indexes**

### WHEN TO RECOGNIZE IT

When maximizing distance between a left value and an equal-or-larger right value, and most left indexes can never beat an earlier smaller candidate.

### VISUAL HOOK

**candidate starts: index 0 → 6**

**new record low: index 1 → 0**

**scan right endpoints from far right**

**index 5 value 5 matches index 1 value 0**

**width = 5 - 1 = 4**

1. Keep indexes of strict new lows while scanning left to right.
2. Scan possible end indexes from the far right toward the left.
3. Pop every start that the current end can satisfy; this is its widest match.

Only record-low values can be useful left endpoints: any later, larger left value is dominated by an earlier smaller one. Scanning right endpoints backward guarantees the first match for a candidate start is its widest possible match.

### CORE MOVE

Build a stack of indexes whose values form strict new lows from left to right. Then scan indexes from right to left; while the current value can pair with the stack top, update the width and pop that start permanently.

### CODE SKELETON

```
starts = stack of strict record-low indexes
for i from left to right:
  if nums[i] < nums[starts.top]:
starts.push(i)
maxWidth = 0
for j from right to left:
  while starts not empty and nums[starts.top]
<= nums[j]:
    maxWidth = max(maxWidth, j -
starts.pop())
return maxWidth
```

### BRUTE FORCE / BASELINE

Try every pair i < j and keep the widest pair where nums[i] <= nums[j]. This directly follows the definition but takes O(n²) time.

### ALTERNATE APPROACHES

- Sort indexes by their values, then track the smallest index seen and maximize current index - smallest index in O(n log n).
- Build prefix minimums and suffix maximums, then use two pointers to find the widest valid pair in O(n) time and O(n) space.

**BUG MAGNETS**

- The candidate-start stack stores indexes, because width depends on positions.
- Push only strict new lows; an earlier equal value always produces an equal or wider ramp.
- Scan right endpoints from right to left so a popped start has already received its widest possible match.

## #24 SortingStack

Medium

Insert into a sorted helper stack Medium Source

**Before reading: how do you insert the current value into the helper stack without random access?**

**PROBLEM DESCRIPTION**

Sort a stack using only stack operations and one extra stack. The sorted helper stack keeps values in order by temporarily moving larger values back to the input stack while inserting the current value.

**INPUT**

`stack top sequence includes [23, 92, 98, 31, 3, 34]`

**OUTPUT**

`Popping the sorted stack prints 98, 92, 34, 31, 23, 3`

**TIME**

**O(n²)**

**SPACE**

**O(n)**

**DATA STRUCTURES**

**Input stack, temporary sorted stack**

**WHEN TO RECOGNIZE IT**

When you need stack-only sorting and cannot use random access, so each new value must be inserted into the right place by moving blockers away.

**VISUAL HOOK**

**curr = 23** **tmp top 31 > 23 → move 31 back**

**push 23 into tmp** **later 31 returns above it**

1. Pop the current value from the input stack.
2. Move helper-stack values back to input while they block the current value.
3. Push the current value into helper, then let moved values get processed again.

Think insertion sort with stacks. The helper stack is kept ordered; if its top is too large for the current value, move that top back to input until the current value fits.

**CORE MOVE**

Pop one value from input. While the helper stack top is greater than that value, move helper values back to input. Push the current value into helper. Repeat until input is empty.

**CODE SKELETON**

```
helper = empty stack
while input not empty:
  curr = input.pop()
  while helper not empty and helper.top > curr:
    input.push(helper.pop())
  helper.push(curr)
return helper
```

**BRUTE FORCE / BASELINE**

Move stack values into an array or list, sort with a library call, then rebuild the stack. It is simpler, but it ignores the stack-operation constraint.

**ALTERNATE APPROACHES**

- Use recursion to sort the stack by inserting each popped element into a sorted recursive suffix.
- If extra data structures are allowed, use a priority queue or list sort for simpler O(n log n) behavior.

**BUG MAGNETS**

- The helper stack's top order determines what popping prints; be clear whether largest or smallest should come out first.
- Move only while helper top is greater than the current value; using the wrong comparison reverses the sorted direction.
- Values moved back to input are not lost; they will be reprocessed later.

## #25 RemoveDuplicateLetters

Medium

Keep smallest safe unique letters Medium Source LeetCode

**Before reading: when is it safe to remove a larger letter already in the answer stack?**

**PROBLEM DESCRIPTION**

Remove duplicate letters so every distinct lowercase letter appears exactly once, and among all valid results, return the lexicographically smallest string.

**INPUT**

`s = "cbacdcbc"`

**OUTPUT**

`"acdb"`

**TIME**

**O(n)**

**SPACE**

**O(1)**

**DATA STRUCTURES**

**Monotonic stack of letters, last-index array, seen set**

### WHEN TO RECOGNIZE IT

When you need one copy of every character, but choosing a smaller earlier character is allowed only if removed larger characters can still appear later.

### VISUAL HOOK

**c then b** **b < c and c appears later → pop c**

**b then a** **a < b and b appears later → pop b**

**keep a**

1. Know the last index where every letter appears.
2. Skip letters already used in the answer stack.
3. Pop larger stack-top letters only if they appear again later.

The stack is the answer being built. A smaller current letter may kick out larger letters from the end, but only when those kicked letters still have another copy later in the string.

### CORE MOVE

Record each letter's last index. Scan left to right; skip letters already in the stack. Before pushing a new letter, pop larger stack-top letters while they appear again later, then push and mark the new letter seen.

### CODE SKELETON

```
last = last index for each letter
seen = false for each letter
stack = empty
for i, c in s:
  if seen[c]: continue
  while stack not empty
        and stack.top > c
        and i < last[stack.top]:
    seen[stack.pop()] = false
  stack.push(c)
  seen[c] = true
return stack as string
```

### BRUTE FORCE / BASELINE

Generate all subsequences containing each distinct letter exactly once and choose the smallest. It proves the goal but is exponential.

### ALTERNATE APPROACHES

- Use a StringBuilder as the stack and a boolean seen array for easier final output.
- Equivalent problem: Smallest Subsequence of Distinct Characters uses the same monotonic stack rule.

### BUG MAGNETS

- Never pop a larger letter if its last occurrence is already behind you; that would lose the only required copy.
- Skip a character if it is already in the stack, otherwise duplicates enter the result.
- When popping, also clear the seen flag so that letter can be added again later.

# #26 RemoveAllAdjacentDuplicatesII

Medium

Delete runs when count reaches k Medium Source LeetCode

**Before reading: what should happen when the current character makes the top run length exactly k?**

**PROBLEM DESCRIPTION**

Given a string and an integer k, repeatedly remove any group of k equal adjacent characters until no such group remains. Return the final string.

**INPUT**

`s = "deeedbbcccbdaa", k = 3`

**OUTPUT**

`"aa"`

**TIME**

**O(n)**

**SPACE**

**O(n)**

**DATA STRUCTURES**

**Stack of character nodes with running duplicate counts**

**WHEN TO RECOGNIZE IT**

When removing a duplicate block can expose a new adjacent block, so the latest survivor must be checked after every deletion.

**VISUAL HOOK**

**d e e e** **third e reaches k = 3**

**pop previous two e's** **current e is skipped**

**d meets next survivor**

1. Track each survivor character with the length of its current adjacent run.
2. If current matches the top and would reach k, remove the stored run and skip current.
3. Otherwise push current with count one or top count plus one.

The stack remembers the current run length for every survivor. When the next same character would make the run length equal k, remove the stored run and do not push the current character.

**CORE MOVE**

For each character, compare with the stack top. If it matches and count + 1 reaches k, pop the previous k - 1 nodes; otherwise push the current character with incremented count. Different characters start a new count.

**CODE SKELETON**

```
stack = characters with counts
for c in s:
  if stack top has same c:
    if stack.top.count + 1 == k:
      pop k - 1 previous copies
      skip current c
    else:
      push Node(c, stack.top.count + 1)
  else:
    push Node(c, 1)
return stack characters from bottom to top
```

**BRUTE FORCE / BASELINE**

Repeatedly scan the string for any k-length adjacent duplicate run, delete it, and restart. This mirrors the rule but can rebuild the string many times.

**ALTERNATE APPROACHES**

- Store compressed pairs of character and count, then decrement or remove a pair when count reaches k.
- Use a StringBuilder as the stack plus a parallel count array indexed by builder position.

**BUG MAGNETS**

- When count reaches k, the current character also disappears, so pop the previous k - 1 and do not push current.
- After a deletion, newly adjacent characters may form another run, so stack exposure is the whole trick.
- Handle empty input separately if the method is meant to be fully reusable.

## #27 RemoveNodesFromLinkedList

Medium

Delete smaller left-side nodes Medium Source LeetCode

**Before reading: what should happen to earlier stack nodes when a larger current node arrives?**

**PROBLEM DESCRIPTION**

Given a linked list, remove every node that has a node with a strictly greater value somewhere to its right. Return the remaining list in original order.

**INPUT**

`head = [5, 2, 13, 3, 8]`

**OUTPUT**

`[13, 8]`

**TIME**
**O(n)**

**SPACE**
**O(n)**

**DATA STRUCTURES**
**Monotonic decreasing stack of ListNode references**

### WHEN TO RECOGNIZE IT

When a later larger node invalidates earlier smaller nodes, but surviving nodes must stay connected as a linked list.

### VISUAL HOOK

**stack: 5, 2** **current: 13**

**13 pops 2** **13 pops 5** **head becomes 13**

**current: 8** **8 pops 3** **13.next = 8**

1. Keep survivor nodes in decreasing value order.
2. Pop every smaller survivor when a larger current node appears.
3. Reconnect the new survivor tail to current, or move head when none remain.

The stack holds nodes that have not yet seen a greater value on their right. When a larger current node arrives, it removes smaller survivors, then the new stack top is rewired to point to current.

### CORE MOVE

Scan from left to right with a decreasing stack of node references. Pop nodes with smaller values than current. If the stack is empty, current becomes the new head; otherwise set stack top's next to current. Push current.

### CODE SKELETON

```
survivors = stack of nodes
push head
for current in remaining nodes:
  while survivors not empty and
survivors.top.val < current.val:
    survivors.pop()
  if survivors empty:
    head = current
  else:
    survivors.top.next = current
  survivors.push(current)
return head
```

### BRUTE FORCE / BASELINE

For every node, scan all nodes to its right looking for a greater value, then rebuild links for only the survivors. This is simple but takes $O(n^2)$ time.

### ALTERNATE APPROACHES

- Reverse the list, keep nodes that are at least the maximum seen so far, then reverse back.
- Use recursion from the right: process the suffix first, then keep the current node only if it is at least the processed suffix head.

**BUG MAGNETS**

- Pop only strictly smaller values; equal values should remain because the rule needs a greater value.
- When the stack becomes empty, update head to the current node.
- After popping, reconnect the surviving stack top directly to current so removed nodes are skipped.

## #28 DecimalToBinary

Easy

Collect remainders backward Easy Source

**Before reading: why does the first remainder become the last character of the binary answer?**

**PROBLEM DESCRIPTION**

Convert a positive decimal integer into its binary string representation by repeatedly dividing by 2 and collecting remainders.

**INPUT**

`num = 18`

**OUTPUT**

`"10010"`

**TIME**

**O(log n)**

**SPACE**

**O(log n)**

**DATA STRUCTURES**

**StringBuilder used as reversed remainder stack**

**WHEN TO RECOGNIZE IT**

When base conversion produces the least significant digit first, so the collected digits must be read in reverse order.

**VISUAL HOOK**

**18 % 2 = 0** **9 % 2 = 1** **4 % 2 = 0**

**2 % 2 = 0** **1 % 2 = 1**

**collected: 01001 → reverse**

1. Take num % 2 to get the least significant bit.
2. Divide num by 2 to move left through the binary places.
3. Reverse the collected remainders before returning.

Every modulo operation gives the next bit from right to left. The first remainder is the final binary character, so the collected sequence must be reversed before returning.

**CORE MOVE**

While the number is positive, append num % 2, then divide num by 2. Reverse the collected remainders to get the binary representation.

**CODE SKELETON**

```
bits = empty builder
while num > 0:
  bits.append(num % 2)
  num = num / 2
return reverse(bits)
```

**BRUTE FORCE / BASELINE**

Repeatedly compare against powers of two from the largest power downward and build each bit. It works, but requires first finding the highest power and is more bookkeeping.

**ALTERNATE APPROACHES**

- Use an actual stack of remainders and pop it into the output.
- Use Integer.toBinaryString for production code when the point is not practicing conversion.

**BUG MAGNETS**

- Remainders arrive in reverse binary order.
- Handle num = 0 separately if zero is allowed; the current loop returns an empty string for zero.
- Integer division by 2 is what moves to the next higher bit.

## #29 OneThreeTwoPattern

Medium

Middle peak with smaller bookends Medium Source LeetCode

**Before reading: which stored peak can prove that the current value is the middle value of a 132 pattern?**

**PROBLEM DESCRIPTION**

Given an integer array, return true if there are three positions i, j, and k in that order where nums[i] is smaller than nums[k], and nums[k] is smaller than nums[j]. The values do not need to be adjacent.

**INPUT**

```
nums = [9, 11, 8, 10, 7, 9, 6]
```

**OUTPUT**

```
true
Example pattern: 8 < 9 < 10
```

**TIME**

**O(n)**

**SPACE**

**O(n)**

**DATA STRUCTURES**

**Monotonic decreasing stack of value/min-before pairs**

### WHEN TO RECOGNIZE IT

When you need a low-high-middle subsequence: the middle index j must be the largest of the three, while k appears after j but still stays below it.

### VISUAL HOOK

**i: 8** **j: 10** **k: 9**

**stack keeps peaks** **each peak carries left min**

**current tests between them**

1. For each candidate peak, remember the smallest value seen before it.
2. Remove weaker peaks that are smaller than the current value.
3. If current sits between the carried left minimum and the remaining peak, return true.

Picture every possible peak nums[j] carrying the smallest value seen before it. A later current value becomes nums[k] if it fits strictly between that carried minimum and the peak.

### CORE MOVE

Scan left to right. Keep a decreasing stack of candidate peaks, where each entry stores the peak value and the minimum value before that peak. Pop smaller peaks before checking the current value against the next stronger peak. If minBefore < current < peak, the 132 pattern exists.

### CODE SKELETON

```
stack = pairs of (peak, minBeforePeak)
for current in nums:
  previous = stack.top pair
  while stack not empty and stack.top.peak <
current:
    stack.pop()
  if stack not empty and stack.top.minBefore
< current < stack.top.peak:
    return true
  stack.push(current, min(previous.minBefore,
previous.peak))
return false
```

### BRUTE FORCE / BASELINE

Try every triple i, j, k and check whether i < j < k and nums[i] < nums[k] < nums[j]. It is direct but $O(n^3)$, so it cannot handle long arrays.

### ALTERNATE APPROACHES

- Scan from right to left with a decreasing stack and a candidate nums[k] value; return true when a left value is smaller than that candidate.
- Precompute prefix minimums for every j, then scan possible k values with a stack or ordered structure to find a value between prefixMin[j] and nums[j].

### BUG MAGNETS

- The index order matters: nums[k] comes after nums[j], even though its value sits between nums[i] and nums[j].
- All comparisons are strict; equal values do not form a 132 pattern.
- Each stored peak must carry the best smaller value from its left side, not just the previous array value.

# #30 RemoveKDigits

Medium

Pop bigger digits before smaller digits Medium Source LeetCode

**Before reading: why does a smaller incoming digit get permission to delete bigger digits immediately before it?**

**PROBLEM DESCRIPTION**

Given a non-negative integer as a string and an integer k, remove exactly k digits so the remaining number is as small as possible. Return the number as a string, with leading zeros removed unless the answer is zero.

**INPUT**

`num = "765028321", k = 5`

**OUTPUT**

`"221"`

**TIME**

**O(n)**

**SPACE**

**O(n)**

**DATA STRUCTURES**

**Monotonic increasing stack of kept digits**

**WHEN TO RECOGNIZE IT**

When deleting digits should make the earliest possible position smaller, because left-side digits dominate the numeric value.

**VISUAL HOOK**

**7 6 5**

**each smaller digit pops the bigger previous digit**

**0 arrives** **pop while top > 0**

**strip leading zeros later**

**k still left?** **remove from the end**

1. Keep the answer digits in stack order.
2. While the top kept digit is bigger than the current digit and removals remain, pop it.
3. After the scan, remove leftover k digits from the end and trim leading zeros.

The stack is the number you are keeping so far. A smaller incoming digit can delete larger kept digits immediately before it, which lets the smaller digit move into a more important left-side position.

**CORE MOVE**

Scan digits from left to right. While the kept stack is not empty, the top digit is larger than the current digit, and k remains, pop the top. Push the current digit. If removals remain after the scan, pop from the end, then strip leading zeros from the final kept digits.

**CODE SKELETON**

```
kept = stack of digits
for digit in num:
  while kept not empty and kept.top > digit
and k > 0:
    kept.pop()
    k--
  kept.push(digit)
while k > 0:
  kept.pop()
  k--
answer = kept without leading zeros
return answer empty ? "0" : answer
```

**BRUTE FORCE / BASELINE**

Try every way to remove k digits, normalize leading zeros, and keep the smallest numeric string. This is useful for tiny examples but combinatorial for real input.

**ALTERNATE APPROACHES**

- Use StringBuilder as the same stack: delete the last kept character while it is larger than the current digit.
- For very small k, repeatedly remove the first digit that is larger than its next digit. This matches the idea but becomes O(k * n).

**BUG MAGNETS**

- Keep popping while top > current, not just once; one incoming digit may improve several earlier positions.
- If k is still positive at the end, the kept digits are non-decreasing, so remove from the right side.
- Strip leading zeros only after all removals; if nothing remains, return "0".